Name: _____
Surname: _____
Date: _____

Debugging (15 points each)

Instructions

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions

1. The following Python code is intended to print the factorials of numbers starting from 3, incrementing by 1 each time, and continuing until the user decides to stop. However, the code contains **three errors** that prevent it from functioning correctly. Identify and fix these errors.

   - Line 4 raises the error `SyntaxError: invalid syntax`. *(5 points)*
   - The loop should print "`3!=6`" but instead prints "`True`". *(5 points)*
   - After fixing the previous errors, the loop should continue with "`4!=24`", then print "`5!=120`", etc., but instead it prints "`3!=18`", then "`3!=54`"... *(5 points)*

```
1  # Initialize the loop control and factorial variables
2  number = 3
3  factorial = 6
4  continue = True
5
6  while continue:
7      factorial = factorial * number
8      print(f"{number!=factorial}")
9      ask = input("Continue? (yes/no): ")
10     # If the user says "yes", proceed to the next number
11     if ask == "yes":
12         continue = True
13     else:
14         continue = False
```

Answer:

The first error is on line 4: `continue` is a reserved keyword of Python (it is used inside loops), so it cannot be used as a variable name. We have to rename it everywhere it appears (line 6, and inside the `if` and `else`), for instance as `keep_going`. *(+5 points)*

The second error is in the `print()`: inside the curly brackets of the f-string, Python evaluates the expression `number!=factorial`, which is a comparison, and the result is `True`. The curly brackets must be placed around each variable separately: `f"{number}!={factorial}"`. *(+5 points)*

The third error is the logic of the loop. The variable `number` is never increased, and the factorial is multiplied before printing. As we already start with `3!=6`, we must first print,

and only if the user answers “yes” we increase `number` by 1 and then multiply the factorial by this new number. *(+5 points)*

(Optional, fixed code)

```
# Initialize the loop control and factorial variables
number = 3
factorial = 6
keep_going = True

while keep_going:
    print(f"{number}!={factorial}")
    ask = input("Continue? (yes/no): ")
    # If the user says "yes", proceed to the next number
    if ask == "yes":
        number += 1
        factorial = factorial * number
    else:
        keep_going = False
```

2. The following code is intended to create two sets: one containing the original members of a group, and another containing both the original and new members. It then displays the difference between these sets. However, the output is not as expected. Explain the issue and fix the code.

```
group_before = {"Luke", "Han", "Leia"}
# Duplicate the original group and add the new members
group_now = group_before
for new_member in ["Ben", "Arturo"]:
    group_now.add(new_member)

# Compare the groups
print(group_now - group_before)
# Expected output: {"Arturo", "Ben"}
# Actual output: set()  <--- Empty set
```

Answer:

The line `group_now = group_before` does not create a copy of the set. It only creates a second name for the same set object, as sets are mutable. Therefore, adding the new members to `group_now` also adds them to `group_before`, and the difference between a set and itself is the empty set. *(+8 points)*

To fix it, we need to create a real copy of the set, using `group_before.copy()` (or `set(group_before)`). *(+7 points)*

(Optional, fixed code)

```
group_before = {"Luke", "Han", "Leia"}
# Duplicate the original group and add the new members
group_now = group_before.copy()
for new_member in ["Ben", "Arturo"]:
    group_now.add(new_member)

# Compare the groups
print(group_now - group_before)  # {"Arturo", "Ben"}
```

3. The following function should mutliply the elements of an array by 2. However the code shows an unexpected result. Explain the issue and fix the code. *(10 points)*

```
import numpy as np

def double_array(arr: np.array) -> np.array:
    # Multiply each element by 2
    arr_double = arr * 2
    return arr_double

print(double_array([1, 2, 3]))
# Expected output: [2, 4, 6]
# Actual output:   [1, 2, 3, 1, 2, 3]
```

How would `double_array()` work on NumPy arrays of 2 or more dimensions? *(5 points)*

Answer:

The function receives a Python **list**, not a NumPy array. Type hints do not convert anything, and the operator `*` on a list means repetition, not an element-wise product. This is why `[1, 2, 3] * 2` gives `[1, 2, 3, 1, 2, 3]`. *(+5 points)*

We need to turn the input into a NumPy array, either inside the function with `arr = np.array(arr)` before the multiplication, or by calling it with `double_array(np.array([1, 2, 3]))`. *(+5 points)*

For arrays of 2 or more dimensions, it works exactly the same: `arr * 2` is an element-wise operation, so it multiplies every single element by 2 and keeps the shape of the array. For instance, `[[1, 2], [3, 4]]` becomes `[[2, 4], [6, 8]]`. *(+5 points)*

(Optional, fixed code)

```
import numpy as np

def double_array(arr: np.array) -> np.array:
    # Make sure we work with a NumPy array
    arr = np.array(arr)
    arr_double = arr * 2
    return arr_double

print(double_array([1, 2, 3]))  # [2 4 6]
```

4. The following code is intended to replace all negative numbers in the array "`data`" with their absolute values using a boolean mask. However, the code raises an error `ValueError: NumPy boolean array indexing assignment cannot assign 7 input values to the 3 output values where the mask is true`. Explain the issue and fix the code. *(10 points)*

```
import numpy as np

data = np.array([5, -3, 7, -1, 0, -6, 8])
mask = data < 0
data[mask] = -data
print(data)
# Expected output: [5, 3, 7, 1, 0, 6, 8]
# Actual output: ValueError
```

Additionally, modify the code so that only the negative numbers greater than -5 are replaced with their absolute values. In the example above, the new output should be: `[5, 3, 7, 1, 0, -6, 8]` *(5 points)*.

Answer:

The mask `data < 0` has only 3 `True` values (the positions of -3, -1 and -6), so `data[mask]` selects 3 elements. On the right side, however, we are assigning `-data`, which has all 7 elements, and NumPy cannot fit 7 values into 3 positions. *(+5 points)*

We must also select the values with the mask on the right side, so that both sides have the same number of elements: `data[mask] = -data[mask]`. *(+5 points)*

For the second part, we need a mask with two conditions: the numbers must be negative **and** greater than -5. We combine two boolean arrays with the operator `&` (not `and`, which does not work element-wise on arrays), and we must put each comparison between parentheses: `mask = (data < 0) & (data > -5)`. *(+5 points)*

(Optional, fixed code)

```
import numpy as np

data = np.array([5, -3, 7, -1, 0, -6, 8])
mask = data < 0
data[mask] = -data[mask]
print(data)  # [5 3 7 1 0 6 8]

# Second part: only negative numbers greater than -5
data = np.array([5, -3, 7, -1, 0, -6, 8])
mask = (data < 0) & (data > -5)
data[mask] = -data[mask]
print(data)  # [5 3 7 1 0 -6 8]
```

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

5. Random Walk (20 points)

You are given a list of player names and a fixed number of steps X. Each player begins at position 0 and performs a random walk of X steps. Write a function `random_walks()` that simulates this process. At each step, the player should either move $+1$ or -1, chosen uniformly at random. After completing the walk for each player, return a dictionary where each key is a player's name, and the corresponding value is that player's final position *(15 points)*.

Your function should accept the following arguments:

- `players` (positional): A list or tuple of player names.
- `num_steps` (keyword): The number of steps for each player's random walk (same number for all of them), defaulting to 10.
- `random_seed` (keyword): If provided, set the random seed to ensure reproducible results.

Additionally, if a player's name appears multiple times in the input list, append a "-" to subsequent occurrences to ensure that all dictionary keys are unique. For example, if "Alex" appears twice, the keys should be "Alex" and "Alex−". If "Bob" appears three times, the keys should be "Bob", "Bob−", and "Bob−−". *(5 points)*

See the code below for an example of input and output:

```
players = ["Alex", "Alex", "Bob", "Bob", "Bob", "Chia"]
x = 10

print(random_walks(players, num_steps=x))
# Example output (actual results will vary due to randomness):
# {"Alex": 2, "Alex-": 0, "Bob": -1, "Bob-": -3, "Bob--": 0, "Chia": 4}
```

Answer:

The function receives the list of players as a positional argument, and the other two as keyword arguments with default values: `def random_walks(players, num_steps=10, random_seed=None)`.

If a seed was provided, we set it at the beginning of the function with `np.random.seed(random_seed)`. We must check `if random_seed is not None`, because 0 is also a valid seed. *(+5 points)*

We initialize an empty dictionary `results = {}` and loop over the players. For each player we simulate the walk: we generate `num_steps` random steps, each of them either -1 or $+1$, using `np.random.choice([-1, 1], size=num_steps)`. *(+5 points)*

Because the player starts at position 0, the final position is simply the sum of all the steps, `steps.sum()`. We store it in the dictionary using the player's name as key, and after the loop we return the dictionary. *(+5 points)*

To keep the keys unique, before storing the result we check whether the name is already a key of the dictionary. While it is, we append a "-" to the name. This way, the second "Bob" becomes "Bob-" and the third one becomes "Bob–". *(+5 points)*

(Optional, code)

```python
import numpy as np

def random_walks(players, num_steps=10, random_seed=None):
    # Set the seed only if the user gave one
    if random_seed is not None:
        np.random.seed(random_seed)
    results = {}
    for player in players:
        # Random steps of -1 or +1, and final position
        steps = np.random.choice([-1, 1], size=num_steps)
        final_position = int(steps.sum())
        # Make the key unique
        name = player
        while name in results:
            name = name + "-"
        results[name] = final_position
    return results
```

6. Count Valid Reports (20 points)

You are given a list of reports, each report consisting of a string of single digits separated by spaces, for instance `"1 2 3"`. A report is considered valid if it meets the following conditions:

1. All digits are strictly increasing from start to end.
2. The absolute difference between any two adjacent digits is at least 1 and at most 3.

Write a function `count_valid_reports()` that, given **any** list of reports of **any size**, returns an integer: the number of valid reports. For instance:

```
ls_reports = [
    "1 2 7 8 9",
    "1 3 2 4 5",
    "1 4 4 6 8",
    "1 3 6 7 9"
]

print(count_valid_reports(ls_reports))  # Output: 1
```

The logic for the example above is:

- `"1 2 7 8 9"`: Invalid because `"2 7"` is an increase of 5.
- `"1 3 2 4 5"`: Invalid because `"1 3"` is increasing but `"3 2"` is decreasing.
- `"1 4 4 6 8"`: Invalid because `"4 4"` is not an increase.
- `"1 3 6 7 9"`: Valid because the digits are all increasing by 1, 2, or 3.

Answer:

As we need to check several reports inside a list, the best approach is to use a for loop. We look at one report at a time and apply the same rules to each of them: `for report in ls_reports`. *(+5 points)*

Because we want to count the valid reports, before the loop starts we initialize a counter, `count = 0`. Inside the loop, every valid report increases it with `count += 1`. At the end of the function, after the loop finishes, we return the resulting count. *(+5 points)*

Inside the loop we have a single report, which is a string. We split it by its spaces with `report.split(" ")`, which returns a list of strings, and turn every string into a number with a list comprehension and `int()`: `digits = [int(s) for s in report.split(" ")]`. For instance, `"1 3 6"` becomes `[1, 3, 6]`. *(+5 points)*

Now we start a nested loop, `for idx in range(1, len(digits))`, to compare every digit against the previous one, with `diff = digits[idx] - digits[idx-1]`. Both rules are met if and only if `1 <= diff <= 3`: a difference of at least 1 means strictly increasing, and at most 3 is the maximum allowed jump. Before the nested loop we set `is_valid = True`. If the difference is outside that range, we set `is_valid = False` and `break`, as a single failure is enough. After the nested loop, if `is_valid` is still `True`, we increase the count. *(+5 points)*

Comment: Checking only that the digits increase is not enough, as `"1 2 7 8 9"` is increasing but has a jump of 5. An empty list of reports returns 0, and a report with a single digit has no pairs to fail, so it counts as valid.

(Optional, code)

```python
def count_valid_reports(ls_reports: list[str]) -> int:
    count = 0
    for report in ls_reports:
        # Example: "1 3 6" -> [1, 3, 6]
        digits = [int(s) for s in report.split(" ")]
        is_valid = True
        for idx in range(1, len(digits)):
            diff = digits[idx] - digits[idx - 1]
            if diff < 1 or diff > 3:
                is_valid = False
                break
        if is_valid:
            count += 1
    return count
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.